Teaching Suggestion Logic

Rule-based, non-prescriptive decision support for instructors

1. Activity Overview & Objective

In this activity, instructor-facing teaching suggestions are generated from engagement trends derived by the multimodal agent. Temporal engagement profiles are analyzed to detect classroom patterns such as **sustained disengagement**, **attention drops**, **high-interaction periods**, and **noisy classroom conditions**.

> **Decision-Support Philosophy.** Rule-based logic converts detected temporal patterns into context-aware recommendations. The mechanism is designed to *support* instructional decision-making, not replace it. All suggestions are grounded in observable data trends and presented in a non-prescriptive manner, ensuring transparency and trust.

Objectives

- Detect meaningful engagement patterns from per-student and class-level attention metrics.
- Map each pattern to a tiered notification (**alert**, **warning**, **info**) with an actionable cue.
- Combine real-time feedback with longitudinal instructor and student trends.
- Keep every suggestion traceable to measurable data so instructors retain full pedagogical autonomy.

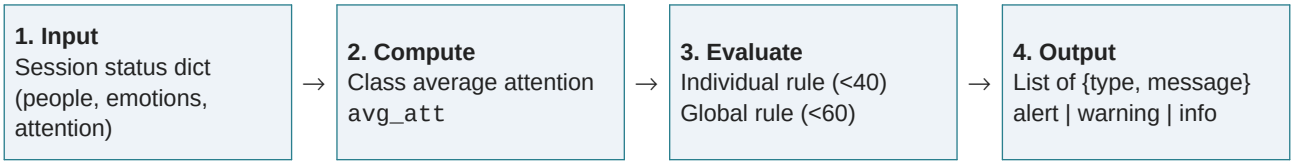
2. Pattern-to-Recommendation Mapping Matrix

By continuously evaluating the temporal engagement profile and multimodal indicators, the engine categorizes classroom dynamics into structured notification tiers paired with actionable, non-prescriptive cues.

Detected Temporal Pattern	Observable Data Trend	Signal Type	Context-Aware Recommendation
Individual Attention Drop	Student attention metric drops below 40% threshold (`p['attention'] < 40`).	**ALERT**	Flags specific student distraction level so the instructor can subtly re-engage them.
Sustained Disengagement	Global class average attention falls below 60% across active participants (`avg_att < 60`).	**WARNING**	Suggests introducing interactive activities, polls, or a brief Q&A segment to restore focus.
High-Interaction / Peak Period	Sustained high engagement (≥ 85%) and balanced emotional diversity across sessions.	**INFO**	Highlights effective teaching segments and strengths to reinforce successful pacing.
Noisy Classroom Conditions	Elevated ambient acoustic energy without structured turn-taking or visual focus.	**WARNING**	Flags classroom management cues to help transition the room back to structured tasks.

Table 1: Mapping of detected classroom patterns to notification tiers and recommendations.

Rule Evaluation Flow



3. Backend Implementation

The teaching suggestion architecture is implemented across two core backend services:

Service	File	Responsibility
TeacherProfileService	`backend/core/teacher_profiles.py`	Aggregates longitudinal effectiveness, session history, and student trends.
RecommendationsEngine	`backend/core/recommendations_engine.py`	Produces immediate rule-based classroom feedback.

3.1 Longitudinal Instructor & Trend Aggregation

`backend/core/teacher_profiles.py` returns an instructor profile containing an effectiveness score, trend delta, class history, per-student trends, and highlighted strengths. The current implementation is a mock; in production it would aggregate average attention across all sessions taught by the instructor.

```
# File: backend/core/teacher_profiles.py
from sqlalchemy.orm import Session
from .database import SessionLocal, Session as SessionModel
import random

class TeacherProfileService:
    def get_profile(self, teacher_id: str):
        # Mock calculation - in production, aggregate real session data
        # Real logic: fetch average attention from all sessions by this teacher
        return {
            "teacher_id": teacher_id,
            "effectiveness_score": 87.5,
            "trend": 12.0,  # +12%
            "class_history": [
                {"name": "CS101 (Today)", "engagement": 85},
                {"name": "CS101 (Yesterday)", "engagement": 78},
                {"name": "Math Workshop", "engagement": 92}
            ],
            "student_trends": [
                {"name": "Priya", "change": 15, "status": "improved"},
                {"name": "Kumar", "change": 8, "status": "improved"},
                {"name": "Asha", "change": -5, "status": "declining"}
            ],
            "strengths": [
                "High engagement sessions (85%+)",
                "Emotion diversity management"
            ]
        }
```

Profile Fields

Field	Meaning
`effectiveness_score`	Overall instructor effectiveness (87.5 in the mock data).
`trend`	Change in effectiveness over time (+12%).
`class_history`	Per-session engagement (e.g. CS101 Today 85, Yesterday 78, Math Workshop 92).
`student_trends`	Per-student change and status (improved / declining).
`strengths`	Reinforceable strengths, e.g. high-engagement sessions and emotion diversity management.

3.2 Real-Time Rule-Based Recommendation Engine

`backend/core/recommendations_engine.py` evaluates a session status dictionary and returns a list of `{type, message}` items, where type is one of **alert**, **warning**, or **info**.

```
# File: backend/core/recommendations_engine.py
class RecommendationsEngine:
    def generate_realtime_recommendations(self, session_status: dict):
        """
        Rule-based recommendations for immediate feedback.
        Input: Session status dict (people, emotions, attention)
        Output: List of { type: 'alert'|'warning'|'info', message: str }
        """
        recs = []

        # 1. Low Attention Alert
        avg_att = 0
        people = session_status.get('people', [])
        if people:
            avg_att = sum(p['attention'] for p in people) / len(people)

        # Individual check
        for p in people:
            if p['attention'] < 40:
                recs.append({
                    'type': 'alert',
                    'message': f"Student {p['id']} is distracted ({p['attention']}%)."
                })

        # 2. Global Attention Warning
        if len(people) > 2 and avg_att < 60:
            recs.append({
                'type': 'warning',
                'message': "Class attention is dropping (<60%). Consider an interactive activity."
            })

        return recs
```

Rule Summary

Rule	Condition	Output
Individual check	`p['attention'] < 40` for any student	**alert**: "Student {id} is distracted ({attention}%)."
Global check	`len(people) > 2 and avg_att < 60`	**warning**: "Class attention is dropping (<60%). Consider an interactive activity."

Note: The global warning requires more than two participants, which avoids misleading class-level signals from very small samples. The high-interaction (info) and noisy-classroom (warning) patterns from the mapping matrix are defined at the design level and are not yet implemented in the code shown.

3.3 Key Engineering Highlights

Observable Data Grounding
Every generated alert or warning traces directly to measurable session metrics: individual attention percentages, global class averages, and longitudinal trend deltas.

Non-Prescriptive Instructor Support
By separating alert, warning, and info cues and pairing real-time telemetry with historical strengths in `TeacherProfileService`, instructors retain full pedagogical autonomy.